

GradeOps Tribunal

Multi-Model Adversarial Grading Pipeline

Technical Implementation Guide · Data Flow · Architecture Reference

1 INGEST

2 EXTRACT

3 TRIBUNAL

4 VERIFY

5 DELIVER

TRIBUNAL

Final Project · AI-Assisted Automated Grading System

Covers: Document Ingestion · Dual Extraction Paths · Adversarial Agent Debate

RAG-Augmented Consistency · CLIP-Based Plagiarism Detection · TA Dashboard

5 Phases

12+ Components

Table of Contents

1.	System Overview & Design Philosophy	3
2.	Phase 1 — INGEST: Document Ingestion Pipeline	4
3.	Phase 2 — EXTRACT: Dual Extraction Paths	5
4.	Phase 3 — THE TRIBUNAL: Adversarial Agent Debate	7
5.	Phase 4 — VERIFY: Coordinator & Consensus Loop	10
5b.	Phase 4B — Parallel Plagiarism Detection (CLIP)	11
6.	Phase 5 — DELIVER: Persistence & TA Dashboard	12
7.	Full Data Flow Reference	13
8.	Technology Stack & Rationale	14
9.	Implementation Guide & Integration Notes	15
10.	Deployment Architecture & Scaling Considerations	17

1. System Overview & Design Philosophy

GradeOps Tribunal is a production-grade, multi-agent AI pipeline designed to automate the grading of handwritten and typed exam papers at scale. The system coordinates three specialist AI agents in an adversarial debate loop — a Grader, a Critic, and a RAG retrieval agent — arbitrated by a lightweight Coordinator. Unlike naive single-model grading, GradeOps is explicitly designed to surface disagreements, catch partial-credit errors, and maintain consistency across an entire cohort by anchoring every decision in semantically similar past grades.

The system simultaneously runs a CLIP-based visual plagiarism detector as a background job, embedding raw answer images (not just their text transcriptions) to catch diagram copies and spatially-similar proofs that text-level similarity metrics miss entirely.

Design Principle	Technical Mechanism	Why It Matters
Multimodal grounding	Raw image crops fed directly to VLMs alongside OCR text	Eliminates OCR translation loss on diagrams, spatial math, and free-body diagrams
Adversarial consistency	Critic agent with devil's-advocate system prompt runs on every graded answer	Forces explicit justification; surfaces partial-credit omissions before persistence
RAG-anchored calibration	ChromaDB / pgvector semantic search over historically graded answers	Grading consistency is a retrieval problem — similar answers should get similar scores
Bounded retry budget	Coordinator enforces a max-iteration cap on the debate loop	Prevents infinite spin in edge cases; guarantees pipeline forward progress
Visual plagiarism	CLIP embeddings + cosine similarity over image vectors	Catches diagram copies and spatial proof reproductions invisible to text diff
Human-in-the-loop	TA dashboard with keyboard-driven approve/override/flag workflow	AI provides the first pass; humans retain final authority on contested grades

Five-Phase Architecture at a Glance: INGEST (document splitting) → EXTRACT (OCR + raw image routing) → TRIBUNAL (parallel agent debate) → VERIFY (coordinator consensus + plagiarism) → DELIVER (database persistence + TA dashboard + gradebook export).

PHASE 1 · INGEST

Document Ingestion & Page Splitting

2. Phase 1 — INGEST: Document Ingestion Pipeline

The pipeline accepts two parallel inputs: a bulk-upload exam scan PDF (potentially containing hundreds of student papers) and a Rubric JSON file that encodes per-question scoring criteria and point weights. These two inputs converge at the PDF Splitter + Cropper, which is the first orchestration component.

2.1 Input: Exam Scan PDF

Scanned exam PDFs arrive as multi-page, multi-student documents. The ingestion layer must handle variable scan quality (300 DPI minimum recommended), skewed pages, and inconsistent answer box boundaries. Each PDF is stored in object storage (e.g., S3 or GCS) immediately on upload and assigned a `batch_id`. A task queue (Celery + Redis) dispatches per-PDF processing jobs.

2.2 Input: Rubric JSON Schema

The rubric is the authoritative scoring contract. It is parsed once per batch and injected into every downstream agent context. A well-structured rubric JSON is critical — ambiguous rubrics directly degrade Grader output quality.

Rubric JSON schema (recommended):

```
{ "exam_id": "CS301-Midterm-2025", "questions": [ { "q_id": "Q1", "max_points": 10, "criteria": [ { "id": "c1", "description": "Correct base case identified", "points": 2 }, { "id": "c2", "description": "Inductive step is logically sound", "points": 5 }, { "id": "c3", "description": "Notation is mathematically precise", "points": 3 } ], "answer_region": { "page": 1, "bbox": [0.05, 0.30, 0.95, 0.75] } } ] }
```

2.3 PDF Splitter + Cropper

Implemented using PyMuPDF (fitz) or pdftplumber, the splitter performs two operations: (1) page-level splitting — separating each student's answer sheet — and (2) question-level cropping — extracting a per-question bounding-box image crop from each page. The bounding boxes are defined in the rubric's `answer_region` field as normalized `[x0, y0, x1, y1]` coordinates relative to the page dimensions.

Splitter pseudocode:

```
import fitz # PyMuPDF def crop_answer_region(pdf_path, page_num, bbox_norm, dpi=150): doc = fitz.open(pdf_path) page = doc[page_num] rect = page.rect # Convert normalised bbox → absolute pixel coordinates clip = fitz.Rect( bbox_norm[0] * rect.width, bbox_norm[1] * rect.height, bbox_norm[2] * rect.width, bbox_norm[3] * rect.height ) mat = fitz.Matrix(dpi/72, dpi/72) # render at target DPI pix = page.get_pixmap(matrix=mat, clip=clip) return pix.tobytes("png") # → raw PNG bytes for downstream agents
```

Inputs

- Exam scan PDF (bulk, multi-student)
- Rubric JSON (per-question criteria + bounding boxes)
- `batch_id` from object storage

Outputs

- Per-question PNG image crops (raw bytes)
- Structured rubric dict (Python object)
- `student_id × question_id` routing map
- Metadata: `page_num`, `crop_bbox`, `dpi`

PHASE 2 · EXTRACT

Dual Extraction: OCR Path vs. Raw Image Bypass

3. Phase 2 — EXTRACT: Dual Extraction Paths

The EXTRACT phase is one of the most architecturally distinctive decisions in GradeOps. Rather than treating OCR as a universal preprocessing step, the system explicitly routes each answer crop down one of two paths based on content type. This routing is determined by the rubric's `content_type` field or by a lightweight content classifier.

3.1 The Routing Decision

A fast, lightweight content-type classifier (a fine-tuned ViT or simple heuristic based on symbol density) examines each crop and assigns it to either the OCR path or the raw image bypass path. The rubric can also explicitly override this with a `bypass_ocr: true` flag per question.

Decision Factor	OCR Path (Nougat / Qwen-VL)	Raw Image Bypass
Content type	Dense handwritten text, formulae, structured prose	Diagrams, graphs, geometric proofs, spatial math, free-body diagrams
Why this path	OCR converts handwriting to structured text the LLM can reason over precisely	OCR would destroy spatial relationships that are semantically meaningful
OCR model used	Nougat (Meta) for LaTeX-aware math parsing; Qwen-VL as fallback for general handwriting	N/A — raw PNG bytes sent directly to VLM's vision encoder
Output format	Markdown + LaTeX string (e.g., $\sum_{i=0}^n i = \frac{n(n+1)}{2}$)	Base64-encoded PNG in VLM multimodal message payload
Information loss risk	Low for text; medium for inline diagrams (OCR may skip)	Effectively zero — VLM sees original pixel data

3.2 OCR Path: Nougat / Qwen-VL

Nougat (Meta, 2023) is a transformer-based document understanding model specifically pre-trained on academic papers and mathematical notation. It converts page images directly to Markdown with embedded LaTeX — making it ideal for exam answers that contain inline equations, summations, integrals, and proof notation. Qwen-VL serves as a general-purpose fallback for answers with heavy freehand prose or non-standard notation.

Nougat inference call (HuggingFace):

```
from transformers import NougatProcessor, VisionEncoderDecoderModel from PIL import Image
import io processor = NougatProcessor.from_pretrained("facebook/nougat-base") model =
VisionEncoderDecoderModel.from_pretrained("facebook/nougat-base") def ocr_crop(png_bytes:
bytes) -> str: image = Image.open(io.BytesIO(png_bytes)).convert("RGB") pixel_values =
processor(image, return_tensors="pt").pixel_values outputs = model.generate(pixel_values,
max_new_tokens=512, bad_words_ids=[[processor.tokenizer.unk_token_id]]) return
processor.batch_decode(outputs, skip_special_tokens=True)[0] # Returns: Markdown string with
LaTeX formulae
```

3.3 Raw Image Bypass Path

For diagrams, circuit schematics, geometric constructions, and spatial math proofs, the PNG crop is sent directly to the Grader VLM without any OCR intermediary. The image is Base64-encoded and injected into the VLM's multimodal message payload as an `image_url` or `image` content block. This preserves every pixel of spatial information that OCR would destroy — relative positions of labels, arrow directions, shaded regions, and geometric relationships.

3.4 Context Assembler

The Context Assembler is the merge node that packages both extraction outputs — OCR transcript (if available) and raw image crop — along with the rubric JSON for the specific question into a single structured context object. This context object is what all three Tribunal agents receive as input.

Assembled context object schema:

```
{ "student_id": "S042", "question_id": "Q3b", "rubric": { "max_points": 8, "criteria": [...] },  
  "ocr_transcript": "Let A be a set. By definition of power set,  $P(A) = \dots$ ", "image_crop_b64":  
  "iVBORw0KGgoAAAANS...", // null if OCR-only question "bypass_ocr": false, "rag_examples": [] //  
  populated by RAG Agent before Grader receives context }
```

PHASE 3 · THE TRIBUNAL

Adversarial 3-Agent Grading Debate

4. Phase 3 — THE TRIBUNAL: Adversarial Agent Debate

The Tribunal is the architectural centrepiece of GradeOps. Three specialist agents execute in parallel — each approaching the same answer crop from a distinct adversarial stance. This multi-agent debate pattern is fundamentally different from ensembling: the agents are not voting on a score, they are constructing arguments that the Coordinator must adjudicate. The adversarial tension is deliberate and load-bearing.

Grader Agent

GPT-4o Vision / Gemini Pro Vision

Primary scoring model. Heavyweight VLM that sees both the raw image crop and the OCR transcript simultaneously, conditioned on the full rubric.

- Receives: image_crop_b64 + ocr_transcript + rubric + rag_examples
- System prompt: 'You are a fair but rigorous examiner. Score each rubric criterion independently. Output structured JSON only.'
- Output schema: { score, max_score, criteria_breakdown: [{id, awarded, justification}], transcribed_snippets: [...] }
- Model choice rationale: GPT-4o and Gemini Pro Vision lead on mathematical OCR benchmarks and can reason over handwritten LaTeX in situ
- Cost profile: ~\$0.01–0.03 per answer depending on image size; budget ~60% of per-answer cost here

Critic Agent

Claude Haiku / Llama 3-8B

Adversarial auditor. Deliberately cheap, fast, and instruction-tuned to find what the Grader missed. Runs on the Grader's output — not the raw context.

- Receives: Grader's full JSON output + original rubric + ocr_transcript
- System prompt: 'You are a devil's advocate. Your ONLY job is to find errors in this grade. Look for: partial credit missed, rubric criteria misread, steps hallucinated, score inconsistencies.'
- Output schema: { agrees: bool, objections: [{criterion_id, issue, suggested_correction}], severity: 'minor'|'major' }
- Model choice rationale: Haiku / Llama 3-8B are 10-20x cheaper than GPT-4o; the Critic does NOT need vision — it reasons over text alone
- Cost profile: ~\$0.0005–0.001 per answer; budget ~10% of per-answer cost

RAG Agent

ChromaDB / pgvector + Embedding Model

Consistency anchor. Retrieves the top-3 semantically similar past graded answers from the vector store and injects them as few-shot examples into the Grader's context.

- Executes BEFORE the Grader so examples are available in context
- Embedding model: text-embedding-3-small (OpenAI) or sentence-transformers/all-mpnet-base-v2
- Query: embed(ocr_transcript + question_id) → cosine similarity search over ChromaDB collection
- Retrieved schema: [{ student_id, ocr_transcript, awarded_score, justification }] × 3
- Purpose: transforms 'is this score right?' into 'is this score consistent with similar past answers?'
- Cold-start: seed with instructor-graded reference answers before first batch run

4.1 Grader Agent — Deep Dive

The Grader is the only agent in the pipeline that receives the raw image crop. It constructs a multimodal prompt that interleaves the rubric criteria, the OCR transcript (as a textual hint, not the ground truth), and the image. The structured JSON output requirement is enforced via constrained decoding (function calling / tool use in GPT-4o, or JSON mode in Gemini). This eliminates free-form output parsing entirely.

Grader prompt construction (Python):

```
import base64, json
def build_grader_prompt(ctx: dict) -> list:
    rubric_text = json.dumps(ctx["rubric"], indent=2)
    rag_block = "\n\n".join([f"PAST EXAMPLE (score {ex['score']}/{ex['max']}): {ex['justification']}" for ex in ctx["rag_examples"]])
    messages = [
        {"role": "system", "content": "You are a rigorous but fair exam grader. Score each rubric criterion independently. Output ONLY valid JSON matching the provided schema."},
        {"role": "user", "content": [
            {"type": "text", "text": f"RUBRIC:\n{rubric_text}\n\nPAST EXAMPLES:\n{rag_block}\n\n"},
            {"type": "text", "text": f"OCR TRANSCRIPT (hint only):\n{ctx['ocr_transcript']}\n\n"},
            {"type": "image_url", "image_url": {"url": f"data:image/png;base64,{ctx['image_crop_b64']}"}}
        ]}
    ]
    return messages
```

4.2 Critic Agent — Adversarial System Prompt Design

The Critic's system prompt is the most deliberately adversarial component in the system. It is explicitly instruction-tuned to disagree, not to validate. Common failure modes it is trained to catch include: (a) hallucinated intermediate steps that the Grader accepted as correct, (b) rubric criteria awarded points that the answer doesn't satisfy, (c) partial credit that should have been awarded but wasn't, and (d) score arithmetic errors across criteria.

Critic system prompt (exact):

```
CRITIC_SYSTEM_PROMPT = """ You are a devil's advocate reviewing an AI-generated exam grade. Your ONLY job is to find errors, inconsistencies, and missed partial credit. For each rubric criterion, ask: 1. Did the Grader actually verify this criterion against the answer, or assume it? 2. Are the awarded points consistent with the justification given? 3. Were any steps or concepts hallucinated (claimed present but not in the answer)? 4. Was partial credit available that was not awarded? Output ONLY valid JSON: { "agrees": bool, "objections": [ { "criterion_id": str, "issue": str, "suggested_score": int, "severity": "minor|major" } ], "overall_severity": "none|minor|major" } """
```

4.3 RAG Agent — Vector Retrieval for Consistency

The RAG agent transforms grading from a one-shot scoring task into a retrieval-augmented few-shot task. By injecting the top-3 most semantically similar past answers (with their awarded scores and justifications), the Grader is implicitly constrained to be consistent across the cohort. This is the primary mechanism for inter-rater reliability at scale.

ChromaDB retrieval (Python):

```
import chromadb from sentence_transformers import SentenceTransformer
embedder = SentenceTransformer("all-mpnet-base-v2")
client = chromadb.PersistentClient(path="./gradeops_db")
collection = client.get_or_create_collection("graded_answers")
def retrieve_rag_examples(ocr_transcript: str, question_id: str, k: int = 3):
    query_embedding = embedder.encode(f"Q:{question_id}\n{ocr_transcript}")
    results = collection.query(query_embeddings=[query_embedding], n_results=k, where={"question_id": question_id})
    # filter to same question only
    return [
        {
            "score": m["awarded_score"],
            "max": m["max_score"],
            "justification": m["justification"],
            "transcript": doc
        } for doc, m in zip(results["documents"][0], results["metadatas"][0])
    ]
```


PHASE 4 · VERIFY

Coordinator Arbitration & Consensus Loop

5. Phase 4 — VERIFY: Coordinator & Consensus Loop

After all three Tribunal agents have produced their outputs, the Coordinator agent reads both the Grader's scoring JSON and the Critic's objection JSON and makes the final binding decision. The Coordinator is implemented on the same cheap, fast model as the Critic (Claude Haiku / Llama 3-8B) because its task is purely textual reasoning over structured data — it has no need for vision capabilities.

5.1 The Consensus Diamond

The consensus decision is a simple binary: did the Grader and Critic agree? Agreement is defined operationally as: (a) **agrees=true** from the Critic, OR (b) all objections have severity='minor' AND the score delta across all criteria is zero. If consensus is reached, the Grader's output is persisted immediately. If consensus is NOT reached, the Coordinator enters arbitration.

5.2 Coordinator Arbitration Logic

The Coordinator receives the full Grader JSON, the Critic objection JSON, and the original rubric. It must produce a final_grade JSON that either accepts the Grader's score, modifies it per the Critic's suggestions, or rejects both and calls for a human-review flag. The Coordinator is the ONLY component that can emit a requires_human flag — this escalates directly to the TA dashboard with high priority.

Coordinator decision schema:

```
{ "final_score": 7, "max_score": 10, "resolution": "grader_accepted" |
  "critic_correction_applied" | "requires_human", "resolution_rationale": "Critic identified
  that criterion c2 awarded 5/5 but the " "inductive step contained an unverified base
  assumption...", "criteria_final": [ { "id": "c1", "awarded": 2, "source": "grader" }, { "id":
  "c2", "awarded": 3, "source": "coordinator", "override_reason": "Critic objection:
  hallucinated step in induction" } ], "iteration": 1, // incremented on each retry
  "requires_human": false }
```

5.3 Retry Budget & Loop Termination

The consensus loop has a hard maximum iteration count (default: 3). On each failed consensus check, the Coordinator's latest resolution is fed back into the Critic as an additional input alongside the original Grader output. If max iterations are exhausted without consensus, the answer is automatically flagged requires_human=true and forwarded to the TA dashboard with iteration history attached.

Loop termination conditions: (1) Critic agrees=true on any iteration, (2) All remaining objections are severity=minor and coordinator accepts them, (3) max_iterations reached → escalate to human review. This guarantee ensures the pipeline never stalls on a single answer.

PHASE 4B · PARALLEL JOB

CLIP-Based Visual Plagiarism Detection

5b. Phase 4B — Parallel Plagiarism Detection (CLIP)

The plagiarism detector runs as an independent background job — fully decoupled from the Tribunal loop. It receives the raw image crops from the EXTRACT phase and embeds them using OpenAI CLIP (Contrastive Language-Image Pre-training) into a shared embedding space. It then computes pairwise cosine similarities across all crops for the same question_id within a batch.

5b.1 Why CLIP and Not Text Similarity?

Text-level similarity (e.g., TF-IDF, BM25, sentence embeddings over OCR output) is blind to visual structure. A student who copies a diagram, a free-body diagram, a circuit schematic, or a geometric proof can trivially avoid text-level detection by relabelling nodes. CLIP embeds the raw pixel content into a semantic vector space where visually similar images (regardless of label text) cluster together. This closes a significant loophole that text-only plagiarism detection leaves open.

CLIP embedding + cosine similarity (Python):

```
import torch import numpy as np from PIL import Image from transformers import CLIPProcessor, CLIPModel
import io clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
clip_proc = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
def embed_crop(png_bytes: bytes) -> np.ndarray:
    image = Image.open(io.BytesIO(png_bytes)).convert("RGB")
    inputs = clip_proc(images=image, return_tensors="pt")
    with torch.no_grad():
        embedding = clip_model.get_image_features(**inputs)
    return embedding[0].numpy() / np.linalg.norm(embedding[0].numpy()) # L2-normalise
def detect_plagiarism(crops: dict, threshold: float = 0.92) -> list:
    """crops: { student_id -> png_bytes }"""
    embeddings = { sid: embed_crop(b) for sid, b in crops.items() }
    flags = []
    ids = list(embeddings.keys())
    for i in range(len(ids)):
        for j in range(i+1, len(ids)):
            cosine_sim = np.dot(embeddings[ids[i]], embeddings[ids[j]])
            if cosine_sim >= threshold:
                flags.append({ "pair": (ids[i], ids[j]), "similarity": float(cosine_sim), "flag": "VISUAL_PLAGIARISM_SUSPECTED" })
    return flags
```

5b.2 Threshold Calibration

The cosine similarity threshold (default 0.92) is a hyperparameter that must be calibrated per exam type. For diagram-heavy exams (circuit analysis, geometry), a threshold of 0.88–0.90 is appropriate since legitimate solutions often share structural similarity. For proof-based exams, 0.93–0.95 reduces false positives. Flags are soft — they add a plagiarism_suspected field to the database record and surface in the TA dashboard for human review; no automatic score penalty is applied.

Implementation note: Run CLIP embedding as a batched GPU job over all crops for a given question_id simultaneously. The pairwise comparison is $O(n^2)$ in the number of students — for cohorts above 200, use approximate nearest-neighbour libraries (FAISS or ScaNN) with an index over the embedding vectors.

PHASE 5 · DELIVER

Database Persistence, TA Dashboard & Gradebook Export

6. Phase 5 — DELIVER: Persistence & TA Dashboard

6.1 Database Persistence

Every resolved grade — whether consensus was immediate or required coordinator arbitration — is written to a PostgreSQL or MongoDB instance. The schema is designed for full auditability: every agent's intermediate output is stored alongside the final grade. This enables post-hoc analysis of inter-agent agreement rates and Critic objection patterns.

PostgreSQL grade record schema (DDL):

```
CREATE TABLE grade_records ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), batch_id TEXT NOT NULL, student_id TEXT NOT NULL, question_id TEXT NOT NULL, -- Final verdict final_score SMALLINT NOT NULL, max_score SMALLINT NOT NULL, resolution TEXT CHECK (resolution IN ( 'grader_accepted', 'critic_correction_applied', 'requires_human')), requires_human BOOLEAN DEFAULT FALSE, human_override BOOLEAN DEFAULT FALSE, -- Agent outputs (JSONB for flexible querying) grader_output JSONB, -- full Grader JSON critic_output JSONB, -- full Critic JSON coordinator_out JSONB, -- Coordinator final JSON rag_examples JSONB, -- top-3 retrieved examples -- Plagiarism plagiarism_flag BOOLEAN DEFAULT FALSE, plagiarism_sim FLOAT, -- cosine similarity score if flagged -- Metadata image_crop_path TEXT, -- S3/GCS URI of the original crop iterations SMALLINT DEFAULT 1, created_at TIMESTAMPTZ DEFAULT NOW(), reviewed_at TIMESTAMPTZ, reviewed_by TEXT );
```

6.2 TA Review Dashboard

The dashboard is a keyboard-driven, high-throughput review interface. Each record is presented as a side-by-side view: the original image crop on the left, the AI-generated grade breakdown on the right. The TA can approve (mark as final), override (manually enter a score with a reason), or flag (escalate for second opinion). The interface is designed for speed: a single keyboard shortcut per action, with the next record auto-loading on action.

Dashboard Feature	Implementation
Side-by-side view	Image crop (S3 presigned URL) + Grader/Coordinator JSON rendered as structured card
Approve (A key)	Sets human_override=false, reviewed_by=TA_id, reviewed_at=now()
Override (O key)	Opens score input modal; writes corrected score + reason; sets human_override=true
Flag (F key)	Sets requires_human=true; routes to senior TA queue
Priority queue	requires_human=true records shown first; sorted by plagiarism_flag, then by score delta (Grader vs Critic)
Bulk approve	For records where Critic agreed and similarity to RAG example > 0.85, allow batch approval
Audit trail	Every action logged to grade_audit_log table with timestamp, TA ID, and before/after state

6.3 Final Gradebook Export

Once a batch reaches the configured `review_threshold` (default: 100% of `requires_human` records reviewed, plus a random sample of auto-approved records), the gradebook export is triggered. The export queries the `grade_records` table, aggregates per-student totals, and outputs CSV / Excel / LMS-format (Canvas, Blackboard, Moodle) files. The export includes a per-student grade breakdown, flagged plagiarism pairs, and a batch-level inter-agent agreement report.

7. Full Data Flow Reference

The table below documents every significant data handoff in the pipeline, the format of the payload at each stage, and the consuming component.

From	To	Payload Type	Key Fields
Exam scan PDF (object store)	PDF Splitter	S3 URI + batch_id	batch_id, student_id, pdf_s3_uri
Rubric JSON (file upload)	PDF Splitter + all agents	JSON object	exam_id, questions[].criteria, questions[].answer_region
PDF Splitter	Content Router	PNG bytes + metadata	student_id, question_id, png_bytes, page_num, crop_bbox_norm
Content Router	Nougat OCR	PNG bytes	question_id, png_bytes (text-heavy answers)
Content Router	Raw Image Path	PNG bytes	question_id, png_bytes (diagram/spatial answers)
Nougat OCR	Context Assembler	Markdown + LaTeX string	ocr_transcript: str (may include \$...\$ LaTeX blocks)
Raw Image Path	Context Assembler	Base64 PNG string	image_crop_b64: str bypass_ocr: true
RAG Agent (ChromaDB)	Context Assembler	List of 3 past grades	rag_examples: [{score, justification, transcript}]
Context Assembler	Grader Agent	Multimodal context object	rubric, ocr_transcript, image_crop_b64, rag_examples, student_id, question_id
Grader Agent (GPT-4o)	Critic Agent + Coordinator	Structured JSON	score, criteria_breakdown, transcribed_snippets, justification
Critic Agent (Haiku)	Coordinator	Objection JSON	agrees: bool, objections: [...], overall_severity
Coordinator (Haiku)	Consensus check	Resolution JSON	final_score, resolution, criteria_final, requires_human, iteration
Consensus: YES	PostgreSQL / MongoDB	Full grade record	All agent JSONs + final_score + plagiarism_flag
Consensus: NO (< max iter)	Coordinator (retry)	Previous resolution	Prior coordinator JSON appended to Critic context
Consensus: NO (max iter hit)	TA Dashboard (high priority)	Escalation record	requires_human=true + full iteration history
CLIP Detector (background)	PostgreSQL	Plagiarism flags	pair: (student_id_a, student_id_b), similarity: float, flag: str
PostgreSQL	TA Dashboard	Grade records (REST API)	Paginated JSON; priority-sorted by requires_human, plagiarism_flag

TA Dashboard	PostgreSQL	Review actions	<code>reviewed_by, reviewed_at, human_override, corrected_score, reason</code>
PostgreSQL	Gradebook export	Aggregated CSV / Excel	<code>student_id, total_score, per_question, flags, batch_agreement_rate</code>

8. Technology Stack & Rationale

Component	Technology / Model	Why This Choice	Alternative
VLM Grader	GPT-4o Vision / Gemini Pro Vision	State-of-art on mathematical handwriting OCR benchmarks; native function calling for structured JSON output	Claude 3.5 Sonnet (vision capable; strong on reasoning tasks)
Critic + Coordinator	Claude Haiku / Llama 3-8B	10-20x cheaper than GPT-4o; task is purely textual — no vision required; low latency for fast loop iterations	Mistral-7B, Gemini Flash
Handwriting OCR	Nougat (Meta / facebook/nougat-base)	Specifically trained on scientific documents with LaTeX; outputs structured Markdown+LaTeX directly	Qwen-VL (stronger on general handwriting); Tesseract (traditional, lower accuracy)
Vector Store (RAG)	ChromaDB / pgvector	ChromaDB: zero-config, Python-native; pgvector: reuses existing Postgres, strong for production	Weaviate, Pinecone, Qdrant
Embedding Model	text-embedding-3-small (OpenAI) / all-mpnet-base-v2	OpenAI: high quality, low cost, managed; all-mpnet: fully local, no external dependency	Cohere embed-v3, BGE-large
Visual Plagiarism	CLIP (openai/clip-vit-base-patch32)	Joint vision-language embedding space; visually similar images cluster regardless of text labels	ResNet-50 embeddings, DINOv2
ANN Index (large batches)	FAISS (Facebook AI)	Sub-millisecond approximate nearest-neighbour at scale; GPU-accelerated; battle-tested in production	ScaNN, Hnswlib
PDF Processing	PyMuPDF (fitz)	Fastest Python PDF library; precise bounding-box crop control; handles encrypted/compressed PDFs	pdfplumber, pdf2image + Pillow
Task Queue	Celery + Redis	Mature, battle-tested; supports per-task priority queues needed for requires_human escalation	RQ, Dramatiq, Temporal
Database	PostgreSQL + JSONB	JSONB columns store agent outputs flexibly; relational structure for cohort-level analytics; pgvector extension for vector ops	MongoDB (document-native but weaker relational analytics)
API Layer	FastAPI (Python)	Async-native; OpenAPI docs auto-generated; Pydantic validation matches JSON schemas used throughout pipeline	Django REST Framework, Flask
Object Storage	AWS S3 / GCS	Industry standard; presigned URLs enable secure direct access from TA dashboard browser	MinIO (self-hosted S3-compatible)

9. Implementation Guide & Integration Notes

9.1 Project Structure

```
gradeops/
├── ingestion/
│   ├── pdf_splitter.py # PyMuPDF-based page + question crop
│   ├── rubric_parser.py # Rubric JSON schema validation (Pydantic)
│   ├── content_router.py # OCR vs. bypass routing logic
│   ├── extraction/
│   │   ├── nougat_ocr.py # Nougat / Qwen-VL OCR wrappers
│   │   ├── raw_image.py # Base64 encoding for bypass path
│   │   └── context_assembler.py # Merge OCR + image + rubric + RAG
│   └── tribunal/
│       ├── grader_agent.py # GPT-4o / Gemini Vision API calls
│       ├── critic_agent.py # Haiku / Llama adversarial critique
│       ├── rag_agent.py # ChromaDB retrieval + embedding
│       ├── coordinator.py # Consensus check + arbitration loop
│       ├── verify/
│       │   ├── consensus.py # Consensus logic + retry budget
│       │   ├── plagiarism/
│       │   │   ├── clip_embedder.py # CLIP embedding pipeline
│       │   │   ├── similarity.py # Pairwise cosine + FAISS index
│       │   │   └── delivery/
│       │   │       ├── db_writer.py # PostgreSQL JSONB persistence
│       │   │       ├── dashboard_api.py # FastAPI REST endpoints for TA UI
│       │   │       ├── export.py # Gradebook CSV/Excel/LMS export
│       │   │       ├── tasks/
│       │   │       └── celery_app.py # Celery task definitions + routing
│       │       ├── config/
│       │       └── settings.py # Model endpoints, thresholds, API keys
```

9.2 Environment Configuration

```
# .env OPENAI_API_KEY=sk-... ANTHROPIC_API_KEY=sk-ant-... GOOGLE_AI_API_KEY=... # Model selection (swap without code changes) GRADER_MODEL=gpt-4o
CRITIC_MODEL=claude-haiku-4-5-20251001 EMBEDDING_MODEL=text-embedding-3-small # Thresholds
MAX_DEBATE_ITERATIONS=3 PLAGIARISM_THRESHOLD=0.92 RAG_TOP_K=3 # Infrastructure
REDIS_URL=redis://localhost:6379/0 DATABASE_URL=postgresql://user:pass@localhost/gradeops
CHROMA_PERSIST_PATH=./chroma_db S3_BUCKET=gradeops-crops
```

9.3 Celery Task Chain

Each question-level crop is processed as a Celery task chain. The chain enforces ordering (RAG retrieval → Grader → Critic → Coordinator) while allowing the plagiarism detection task to execute concurrently as an independent chord.

```
from celery import chain, chord, group from tasks import (retrieve_rag, run_grader, run_critic,
run_coordinator, run_plagiarism, save_grade) def process_question(student_id, question_id,
context): # Main grading chain (sequential dependencies) grading_chain = chain(
retrieve_rag.s(context), run_grader.s(), run_critic.s(), run_coordinator.s(max_iter=3),
save_grade.s(student_id, question_id), ) # Plagiarism runs in parallel (independent of grading
chain) plagiarism_task = run_plagiarism.s(context["image_crop_b64"], question_id, student_id)
# Launch both grading_chain.delay() plagiarism_task.delay()
```


10. Deployment Architecture & Scaling

10.1 Per-Answer Cost Breakdown

Component	Model / Service	Approx. Cost per Answer	Notes
Grader Agent	GPT-4o Vision	\$0.015–0.030	Depends on image size (token count). Largest cost centre.
Critic Agent	Claude Haiku	\$0.0003–0.001	Text-only; very low token count. ~20-50x cheaper than Grader.
Coordinator	Claude Haiku	\$0.0002–0.0005	Structured JSON comparison; minimal tokens.
OCR (Nougat)	Self-hosted GPU inference	\$0.001–0.003	A100 GPU instance: ~100 crops/min. Amortised over batch.
CLIP Embedding	Self-hosted GPU inference	\$0.0001–0.0005	Shared GPU with Nougat; CLIP is very fast (~500 imgs/sec on A100).
Embedding (RAG)	text-embedding-3-small	\$0.00002–0.0001	Very low cost. 512-dim vectors, short inputs.
Total (no retry)	—	~\$0.017–0.035	Roughly \$17–35 per 1,000 answers.
Total (1 retry avg)	—	~\$0.019–0.040	Critic + Coordinator cost is marginal; retry overhead is small.

10.2 Scaling Considerations

Horizontal Celery Workers

Each Celery worker can process independent question crops in parallel. Scale worker count linearly with batch size. At 500 students × 10 questions = 5,000 crop tasks, 20 workers process a batch in under 30 minutes (assuming 3 min/task including retries).

GPU Inference Server

Host Nougat and CLIP on a shared A100/H100 instance behind an internal HTTP endpoint. Use NVIDIA Triton Inference Server for concurrent model serving and dynamic batching. For cloud deployment, use Modal, Replicate, or HuggingFace Inference Endpoints.

Vector Store Sharding

ChromaDB collections should be partitioned by question_id to keep retrieval within a single question's answer space. For > 50k historical records per question, migrate to pgvector with HNSW index for sub-10ms retrieval.

LLM Rate Limits

GPT-4o has tier-based RPM/TPM limits. Implement exponential backoff with jitter on all LLM API calls. For large batches, use OpenAI Batch API (async, 50% cost reduction) when real-time latency is not required.

Database Connection Pooling

Use PgBouncer in transaction-pooling mode in front of PostgreSQL. Celery workers create many short-lived connections — pooling is essential at scale.

10.3 Monitoring & Observability

Instrument every agent call with OpenTelemetry traces. Key metrics to track: per-question Grader latency (p50/p95), Critic objection rate (objections per 100 answers), Coordinator override rate (how often the Critic's corrections are applied), consensus loop iteration distribution, plagiarism flag rate per batch, and TA review throughput (answers reviewed per TA per hour).

Key success metrics for GradeOps Tribunal: Inter-agent agreement rate (target >80% first-pass consensus), Grader-vs-human correlation on sampled answers (target $r > 0.92$), Plagiarism false-positive rate (target <2%), TA override rate (target <5% of auto-approved records), End-to-end batch latency per 100 students (target <15 min).

GradeOps Tribunal — Final Project Technical Documentation · 5 Phases · 12+ Components · 3 AI Agents · CLIP Plagiarism Detection